



TAC6223

APPLIED CRYPTOGRAPHY

Group Assignment

Trimester March/April 2025 (Term 2510)

Group Name: Ace Cipher

No.	Student ID	Student Name	Major
1.	243UT246XG	ONG YONG QUAN	ST
2.	251UT2512H	CHEE JIONG KING	ST
3.	242UT246XT	GOH XIN RAN	ST

Task Distribution Form

No.	Student ID	Student Name	Tasks Completed
1.	243UT246XG	ONG YONG QUAN	Part 2: Java Implementation of the RSA Signature Scheme
2.	251UT2512H	CHEE JIONG KING	Part 1: Theory and Applications
3.	243UT246XT	GOH XIN RAN	Part 2: Java Implementation of the RSA Signature Scheme

Declaration by the Group

I hereby declare that the assignment was carried out as the results of our own works, and no part of this assignment has been copied from any other student's work or from any other source, except otherwise indicated or acknowledged as referenced works.

Date of Submission: 9/6/2025

Marking Rubric

CLO3: Demonstrate a simple cryptographic scheme for a given security goal.

Assessment Criteria	Excellent	Good	Fair	Poor	Marks
Documentation (8%)	Clearly and effectively documented with excellent descriptions and discussions (7-8)	Good documentation with good descriptions and discussions (5-6)	Basic documentation with fair descriptions and discussions (2-4)	Most or all documentation is missing (0-1)	
Programme execution (7%)	Programme executes correctly with no syntax or runtime errors (6-7)	Programme executes with very minor/insignificant errors (4-5)	Programme executes with slightly major errors (2-3)	Programme does not execute (0-1)	
Correct output and user interface (5%)	Output is correctly displayed with no errors and good design of user interface (5)	Output has minor errors and displays minimally expected design of user interface (4)	Output has multiple errors and does not display the required user interface (2-3)	Output is incorrect with poorly designed user interface (0-1)	
TOTAL					

Part 1: Theory and Applications

Digital signature is a secure method which is used to confirm the authenticity and integrity of digital data, including documents, messages and software. With digital communication, It can help to prevent tampering and impersonation. Besides, it can also provide evidence of origin, identity and status of electronic documents, transactions and digital messages. Digital signatures are commonly used in activities like online banking and software update distribution to protect sensitive actions.

1.1 Online Banking Transaction

In online banking transactions, the key security properties required are:

- Data Origin Authentication: Confirm that the transaction request truly originates from the authorized account holder.
- Data Integrity: Confirm that the transaction data, such as the amount and recipient account, are not modified during transmission.
- Non-repudiation: Prevent the user from later denying that they initiated the transaction.

Digital signatures are used in online banking to guarantee secure and authentic transactions like fund transfer. First, a bank customer as the sender generates a hash of the transaction details using a secure hash function, like SHA-512. After that, This hash is encrypted with the sender's private key, producing the digital signature. The original message and the digital signature, which are signed messages, are sent to the bank. The bank receives the message, it decrypts the signature using the sender's public key to retrieve the original hash, and compares it with a hash of the message that was recently computed. If the hashes match, the message is verified to be authentic, unmodified, and actually from the authorized sender.

1.2 Software Update Distribution

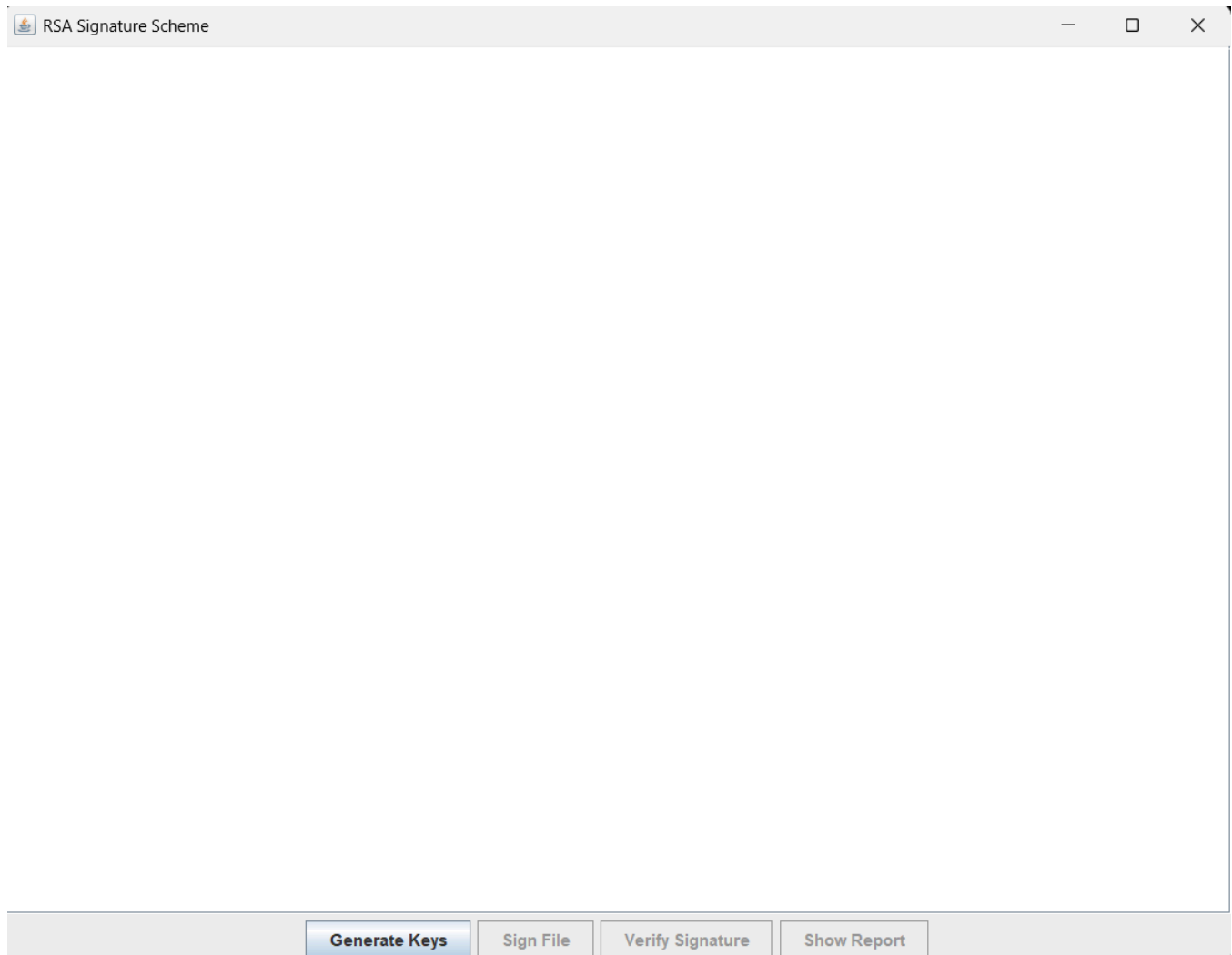
In software update distribution, especially for operating systems and critical applications, the required security properties are:

- Data Origin Authentication: Verify that the update was released by the genuine software provider.
- Data Integrity: Ensure that the software file has not been modified or tampered with.
- Non-repudiation: Prevents the vendor from denying they released the specific version of the update.

When distributing software updates, it is critical to make sure that the update comes from a trusted source and has not been modified or tampered with. First, a software provider generates a hash of the update file and encrypts it using their private key to produce a digital signature. The update file and its digital signature are sent to users. After that, the user's system decrypts the signature using their public key and obtains the original hash. It also computes the hash of the received update file. If the two hash values are identical, the update is verified to be authentic, unmodified, and accountable. Data origin authentication, data integrity and non-repudiation guarantees are provided by this use of digital signatures in software distribution, which also protects users from fake or malicious updates.

Part 2: Java Implementation of the RSA Signature Scheme

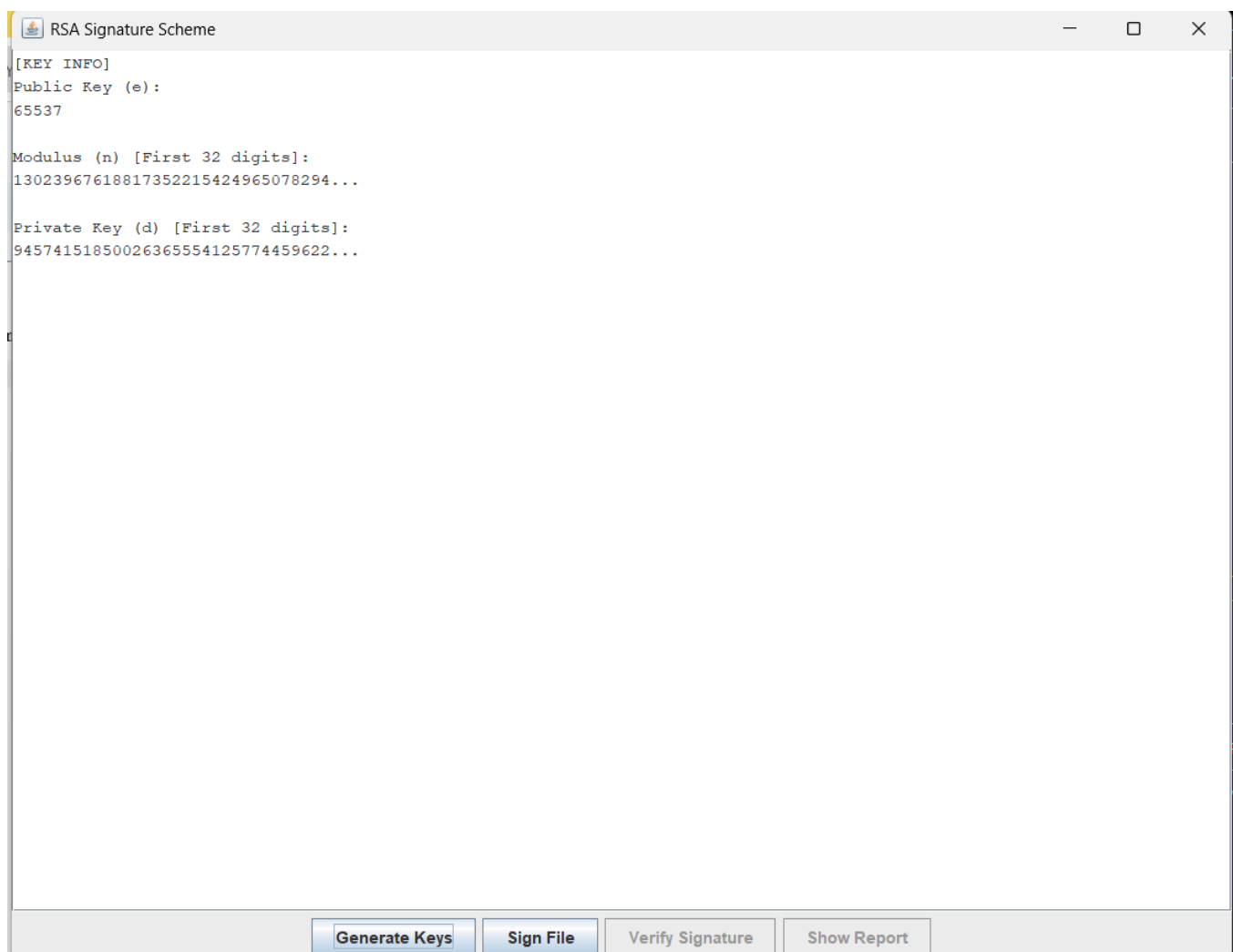
Step 1: It is a RSA Signature Scheme. Click “Generate Keys” Button. The program will initialize the RSA signature scheme.



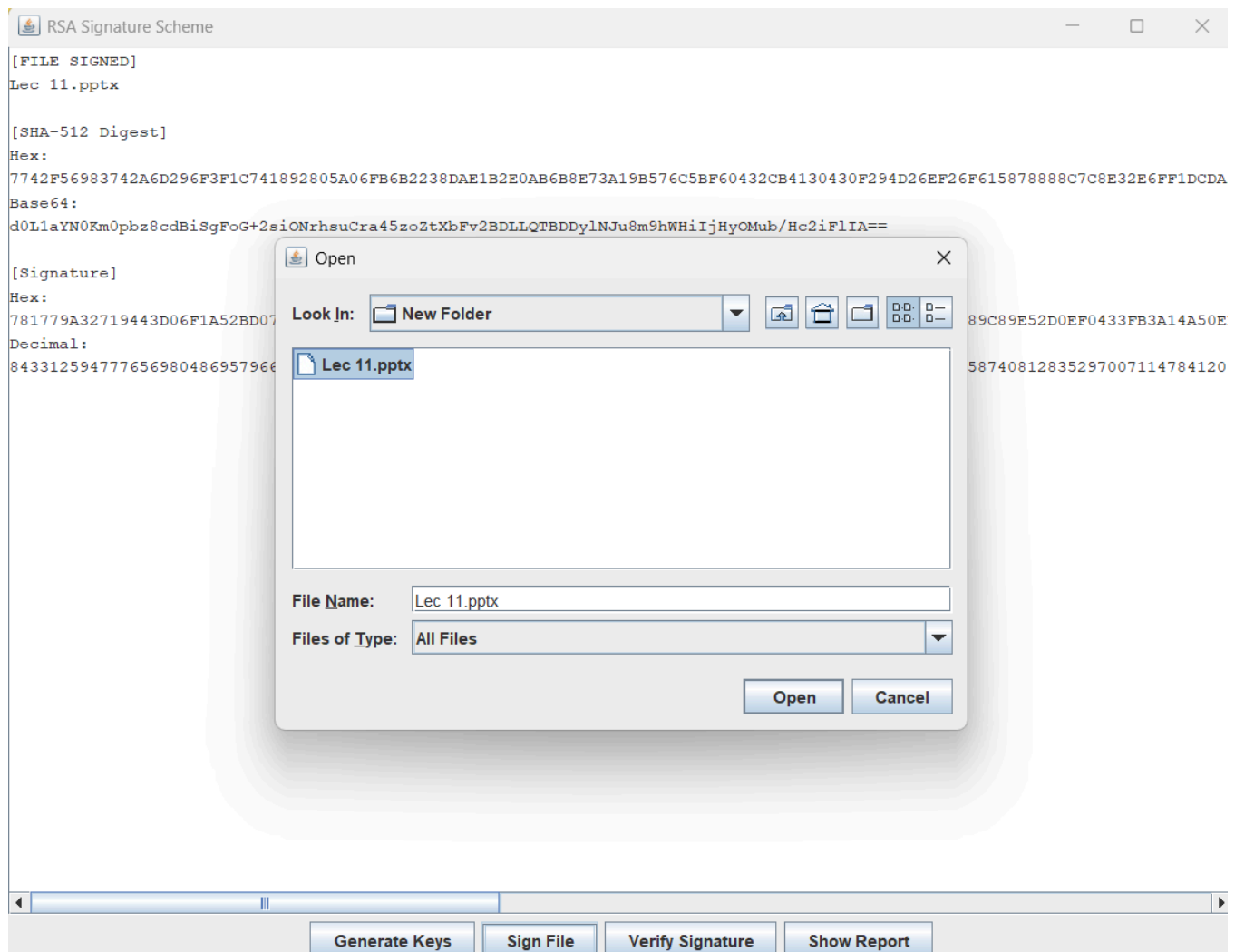
Step 2: Upon clicking the button, the program will:

- Generate two 512-bit prime numbers (p and q)
- Compute the modulus: $n = p \times q$
- Set the public exponent $e = 65537$
- Calculate the private key d as the modular inverse of $e \bmod \phi(n)$

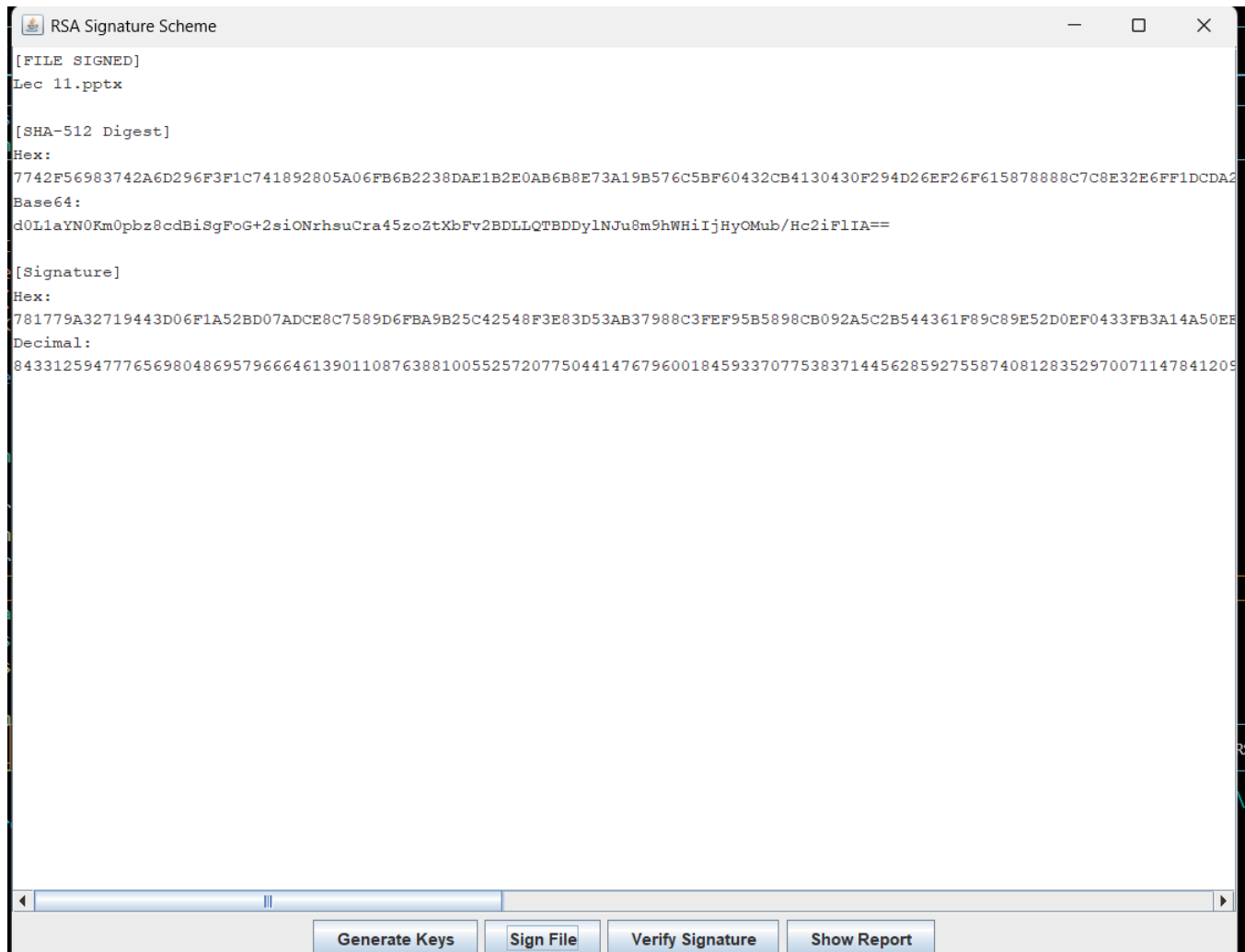
At this point, the RSA key pair is ready



Step 3: Click the “Sign File” button to open the file chooser. Select any file you want to digitally sign.



Step 4: The program computes a 512-bit message digest of the file using the SHA-512 hash function, then generates the RSA digital signature using the formula: **signature = digest^d mod n**. Both the digest and the signature are displayed in **Hexadecimal** and **Decimal** formats.



```

RSA Signature Scheme

[FILE SIGNED]
Lec 11.pptx

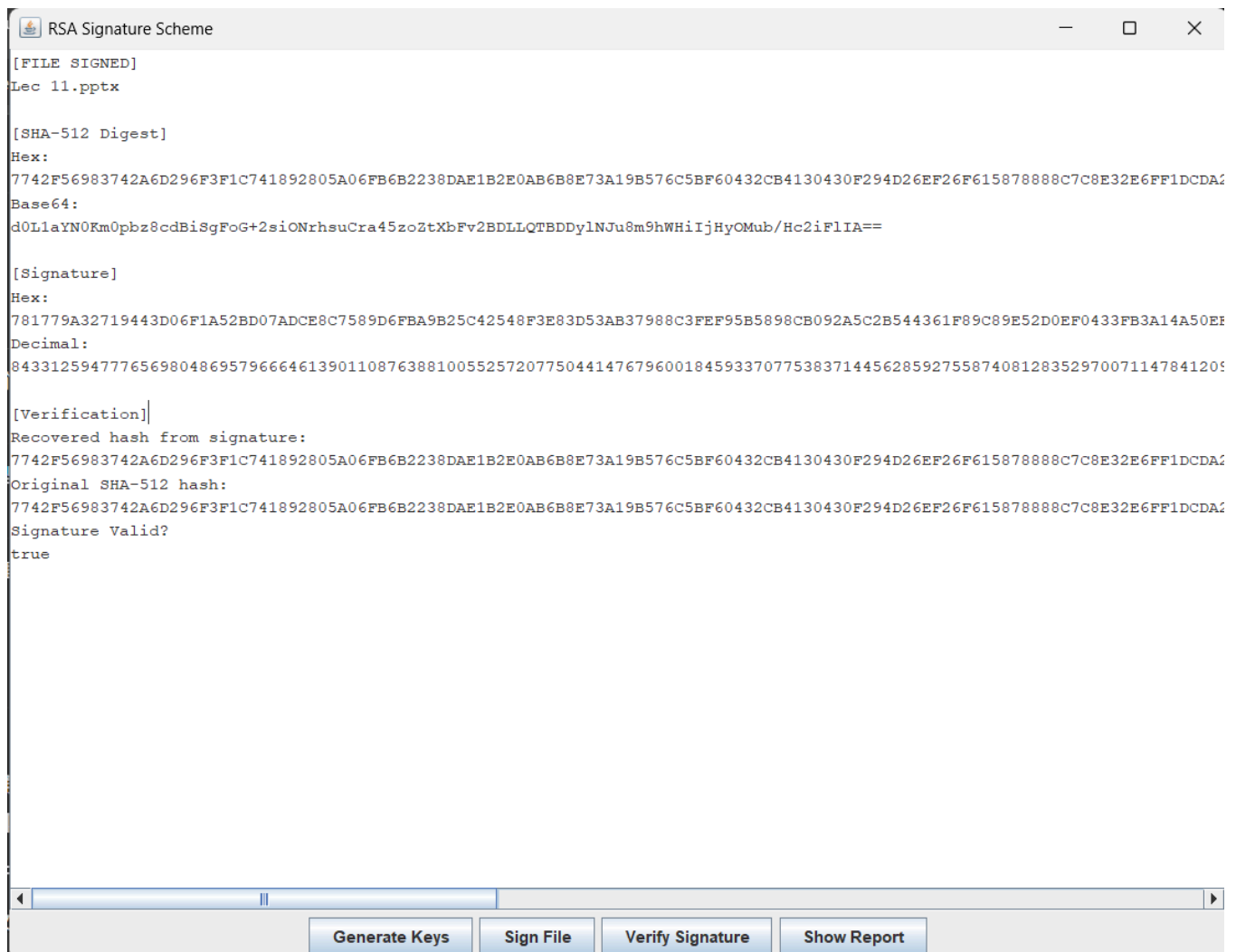
[SHA-512 Digest]
Hex:
7742F56983742A6D296F3F1C741892805A06FB6B2238DAE1B2E0AB6B8E73A19B576C5BF60432CB4130430F294D26EF26F615878888C7C8E32E6FF1DCDA2
Base64:
d0L1aYN0Km0pbz8cdBiSgFoG+2siONrhsuCra45zoZtXbFv2BDLLQTBDdYlNJm9hWHIjHyOMub/Hc2iFlIA==

[Signature]
Hex:
781779A32719443D06F1A52BD07ADCE8C7589D6FBA9B25C42548F3E83D53AB37988C3FEF95B5898CB092A5C2B544361F89C89E52D0EF0433FB3A14A50EE
Decimal:
843312594777656980486957966646139011087638810055257207750441476796001845933707753837144562859275587408128352970071147841209

```

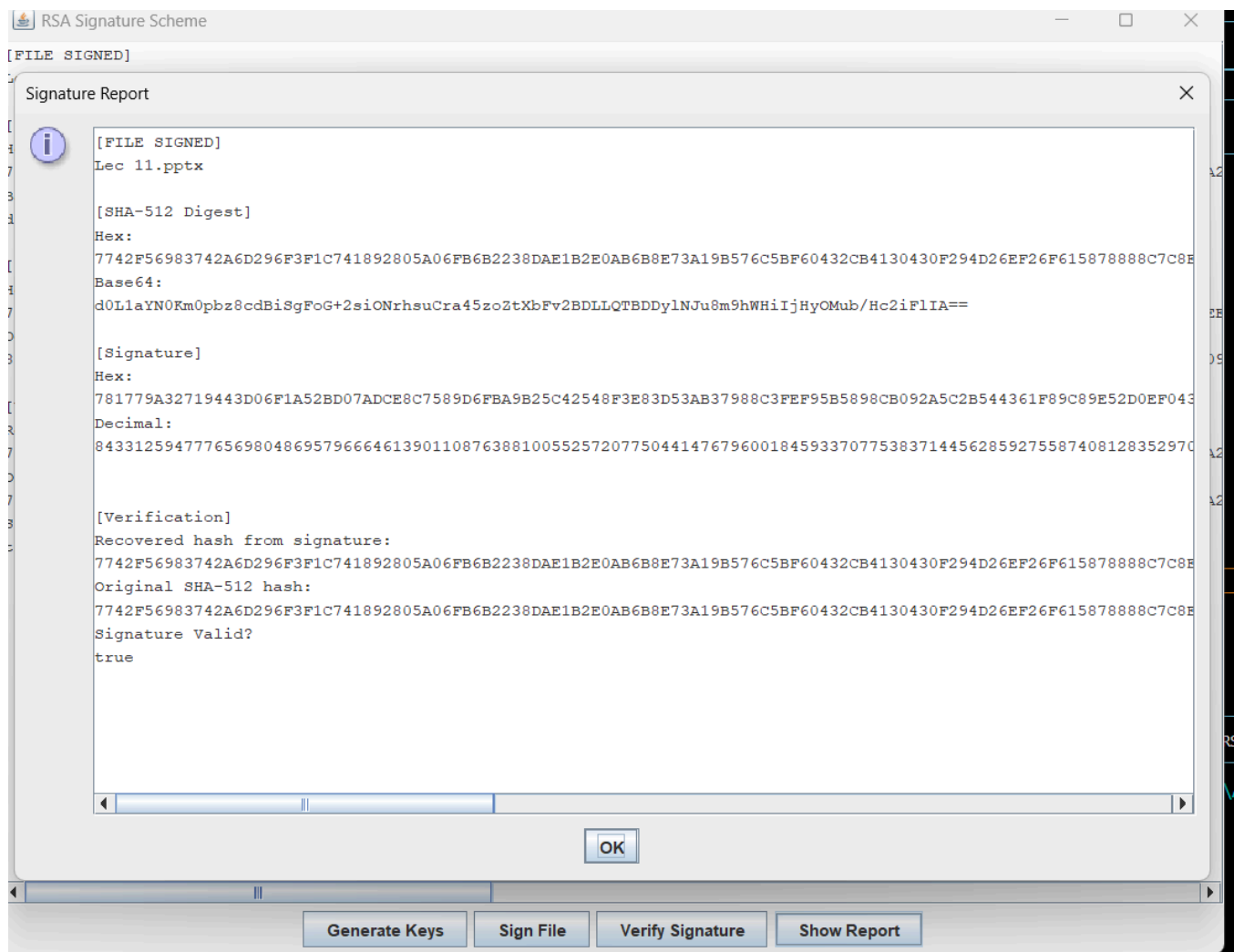
Generate Keys Sign File Verify Signature Show Report

Step 5: Click “**Verify Signature**” to check if the digital signature is valid. The program compares the decrypted signature with the original SHA-512 hash.



Step 6: Click “**Show Report**” to view the saved signature and verification results from Signature_Report.txt , including:

- File name
- SHA-512 hash
- Signature (Hex & Dec)
- Verification result (Valid / Invalid)



Step 7: Once the report is displayed, the program automatically saves a copy to your computer for future reference.

```
Signature_Report.txt
File Edit View

[FILE SIGNED]
Lec 11.pptx

[SHA-512 Digest]
Hex:
7742F56983742A6D296F3F1C741892805A06FB6B2238DAE1B2E0AB6B8E73A19B576C5BF60432CB4130430F294D26EF26F615878888C7C8E32E6FF1DCDA216520
Base64:
d0L1aYN0Km0pbz8cdBiSgFoG+2siONrhCuCra45zoZtXbFv2BDLLQTBDDylnJu8m9hwhI1jHyOMub/Hc2iFlIA==

[Signature]
Hex:
781779A32719443D06F1A528D07ADCE8C7589D6FBA9B25C42548F3E83D53AB37988C3FEF95B5898CB092A5C2B544361F89C89E52D0EF0433FB3A14A50EE81BA1D81ACD2E60EFD49A3DFD721BB1AE0B0CE1A8C7AFA9C29E3D0C30BF08795A40F7C16C731227CFE9FD798E4675F55D3AAFD70DA3BA23945FC517C6DF21850FB20
Decimal:
84331259477765698048695796664613901108763881005525720775044147679600184593370775383714456285927558740812835297007114784120900013490763455204463393284677964558346027363122057358759237002668241266077918777357904774022646118102316459237024613056476672737231548561155158594386451454950693485937355900980554693408

[Verification]
Recovered hash from signature:
7742F56983742A6D296F3F1C741892805A06FB6B2238DAE1B2E0AB6B8E73A19B576C5BF60432CB4130430F294D26EF26F615878888C7C8E32E6FF1DCDA216520
Original SHA-512 hash:
7742F56983742A6D296F3F1C741892805A06FB6B2238DAE1B2E0AB6B8E73A19B576C5BF60432CB4130430F294D26EF26F615878888C7C8E32E6FF1DCDA216520
Signature Valid?
true
```